

The AbstractiFy "Anti-Gravity" Checklist

Building an Un-crashable, Frictionless, and Lightweight Academic Engine

This checklist outlines the exact technical mechanisms, libraries, and code architectures required to transition AbstractiFy from a conceptual PRD into a bulletproof, production-grade application.

1. Eliminate Computational Weight (Performance & Latency)

To make the application feel "weightless," responses must be near-instantaneous. You need to implement caching, asynchronous operations, and batch processing.

- **[] Implement Dual-Layer Caching:**
 - **Memory Caching:** Use `streamlit.cache_data` or `functools.lru_cache` for API calls to Semantic Scholar and OpenAlex. If two users query the same thesis, the second load should take 0 seconds.
 - **Persistent Vector Caching:** Save generated embeddings locally on disk (using FAISS's built-in `index.save()` or a lightweight SQLite database) so you never re-embed the same paper abstract twice.
- **[] Asynchronous API Fetching:**
 - Use `httpx` or `aiohttp` combined with Python's `asyncio` to query OpenAlex, Semantic Scholar, and Gemini concurrently.
 - *Impact:* Reduces search and aggregation latency from ~ 8 seconds sequentially down to < 2 seconds in parallel.
- **[] Batch Embedding Generation:**
 - Instead of passing retrieved abstracts one-by-one to your embedding model, pass them as a list to the sentence-transformer: `model.encode(list_of_abstracts, batch_size=16)`. This utilizes vectorized CPU/GPU matrix multiplication efficiently.

2. Structural Fail-Safes (The "Un-crashable" Guardrails)

Since you are relying on free academic APIs, your app must gracefully survive rate limits, missing data, and complex formatting anomalies.

- **[] Exponential Backoff Decorator:**
 - Wrap all external API requests (Gemini, Semantic Scholar) in a retry decorator.

- *Mathematical Rule:* Implement backoff delay $T_{\text{delay}} = 2^{\text{attempt}} + \text{jitter}$ (where jitter is a random float between 0 and 1 to prevent thundering herd problems).
- **[] Graceful Degradation (Fallback Modes):**
 - **No API Key?** Fall back automatically to a local HuggingFace zero-shot classifier for the "Consensus Meter" and a local TF-IDF ranker if FAISS model loading fails.
 - **Abstract Missing?** If a fetched paper has a blank abstract field, write a fallback routine to auto-query the DOI directly or gracefully skip the paper without crashing the table rendering.
- **[] Strict Pydantic Validation & Parsing Protections:**
 - When utilizing Gemini for structured JSON generation, wrap the output parsing in a try-except block.
 - If the LLM generates slightly malformed JSON, pass the error back to a lightweight regex cleaner before failing, or default to a safe `StudyDetails(dataset_or_sample_size="Unknown", ...)` object.

3. Advanced PDF & Text De-jargonization Mechanics

Handling user-uploaded files requires high-precision parsing to extract mathematical syntax and structures accurately.

- **[] Robust Two-Column Layout Reconstruction:**
 - Standard PDF readers merge text horizontally across columns, breaking sentences.
 - **Fix:** Use PyMuPDF with the blocks parameter: `page.get_text("blocks")`. Sort the blocks programmatically by their vertical coordinate (y) and then horizontal coordinate (x) to reconstruct true column order.
- **[] Inline LaTeX Math Sanitizer:**
 - Academic papers use raw Unicode symbols mixed with LaTeX.
 - **Fix:** Implement a regex cleaner that converts standard Unicode math characters (e.g., $\pm$, $\approx$, Σ , α) into standard LaTeX syntax (`\pm`, `\approx`, `\sum`, `\alpha`) so they render perfectly inside your Markdown and math UI blocks.
- **[] Chunking with Overlap:**
 - When splitting long PDFs for the local vector store, use a recursive character splitter with a chunk size of 1000 characters and an overlap of 200 characters. This ensures semantic context (like the premise of an equation) isn't chopped in half at chunk boundaries.

4. Draggable Network Graphs (Frictionless Visualization)

Visualizing references must be highly responsive and lightweight on the client's web browser.

- **[] Graph Depth Boundary & Pruning:**

- Do not pull more than 2 degrees of citation depth from OpenAlex, or your network graph will crash the user's browser with thousands of nodes.
- **Fix:** Limit your visualization to $N \leq 50$ total nodes. Sort papers by their citation count and discard low-impact papers to keep the canvas clean and readable.
- **[] Asynchronous Pyvis Renderer:**
 - Render Pyvis graphs inside a Streamlit components.html container. Enable physics stabilization (stabilization: { iterations: 100 }) on load so the graph doesn't wobble uncontrollably when the user first opens it.

5. Security, Environment, and Deployment Setup

Ensuring the application is secure and easy for anyone to deploy with a single click.

- **[] Secret Token Isolation:**
 - Never hardcode API keys. Use st.secrets (for Streamlit Cloud) or standard environment variables (os.getenv("GEMINI_API_KEY")).
- **[] Session Memory Cleanup:**
 - Ensure that local uploaded PDFs and FAISS indices are tied strictly to the user's browser session using st.session_state. When the user closes the tab, all local vectorized indices are immediately garbage-collected from the server's RAM.
- **[] Dockerization for Self-Hosting:**
 - Provide a simple Dockerfile so users can run AbstractiFy completely offline on their local machines with a single terminal command: docker compose up.